

RECRUITAI

PROJECT SPECIFICATION & IMPLEMENTATION BLUEPRINT

Real Time Collaborative To Do List

Detailed project brief generated from the project record.

CANDIDATE

Tanvi Joshi

DURATION

10 days

TECHNOLOGY STACK

Python

Django

React

REST API

WebSockets

MySQL

Project Overview

Summary of the problem, purpose and expected outcome.

PROJECT SUMMARY

A web app where users can manage personal to do lists and share them with others, seeing updates instantly via WebSockets.

CANDIDATE Tanvi Joshi	TYPE fullstack	DURATION 10 days
---	--------------------------------------	--

Primary Objectives

- Create a responsive React UI for list management.
- Build a Django REST API with JWT authentication.
- Enable real time list updates with Django Channels.
- Persist data in MySQL with proper schema.
- Write unit and integration tests covering core features.

Candidate Details

NAME

Tanvi Joshi

Technology Stack

Technologies, frameworks and tools expected for implementation.

Python Technology / Tool	Django Technology / Tool	React Technology / Tool
--	--	---

REST API

Technology / Tool

WebSockets

Technology / Tool

MySQL

Technology / Tool

Functional Requirements

- Users can register and log in.
- Authenticated users can create, edit, and delete tasks.
- Users can create multiple to do lists.
- A list owner can share a list with other registered users.
- Shared list members see task changes in real time.
- Users can log out.

Non-Functional Requirements

- Passwords stored with strong hashing (PBKDF2).
- All API inputs validated; errors returned with proper status codes.
- UI must be responsive on desktop and mobile browsers.
- Real time updates delivered within 200 ms under normal load.
- Codebase documented with README and API spec.

Modules & Features

Major functional areas that make up the project.

Authentication

Handles user registration, login, JWT issuance, and logout.

- Register endpoint
- Login endpoint
- JWT middleware
- Password hashing

To Do Management

CRUD operations for lists and tasks.

- Create list
- Add/edit/delete task
- List retrieval
- Delete list

Collaboration

Sharing lists and broadcasting changes via WebSockets.

- Share list endpoint
- Permission checks
- WebSocket channel per list
- Broadcast task changes

Frontend UI

React interface for all user interactions.

- Login/register forms
- Dashboard with lists
- Task editor
- Real time update handling

Implementation Roadmap

Recommended phases and milestones for completing the project.

1

Foundation

Initialize Git repo, Create Django project with DRF, Create React app with create react app, Configure MySQL connection, Add ESLint/Prettier, Write initial README

- Initialize Git repo
- Create Django project with DRF
- Create React app with create react app
- Configure MySQL connection
- Add ESLint/Prettier

2

Core Backend

Design DB schema, Create User model and migrations, Implement JWT login/register views, Build List and Task models, Create List/Task viewsets, Add unit tests for models and views

- Design DB schema
- Create User model and migrations
- Implement JWT login/register views
- Build List and Task models
- Create List/Task viewsets

3

Real Time Layer

Install Django Channels, Configure Redis channel layer, Create consumer for list channel, Integrate permission checks, Emit events on task changes, Write integration tests for WebSocket flow

- Install Django Channels
- Configure Redis channel layer
- Create consumer for list channel
- Integrate permission checks
- Emit events on task changes

4

Frontend Development

Set up React router and auth context, Create login/register pages, Build dashboard with list overview, Implement task editor component, Integrate WebSocket client, Add basic responsive styling

- Set up React router and auth context
- Create login/register pages
- Build dashboard with list overview
- Implement task editor component
- Integrate WebSocket client

5

Polish & Delivery

Write end to end tests with Cypress, Add API documentation (Swagger), Create Dockerfile and docker compose for dev, Run linting and fix issues, Prepare final demo video, Update README with usage guide

- Write end to end tests with Cypress
- Add API documentation (Swagger)
- Create Dockerfile and docker compose for dev
- Run linting and fix issues
- Prepare final demo video

API Design

Key API endpoints and expected responsibilities.

POST

/api/auth/register/

Create new user

POST

/api/auth/login/

Obtain JWT token

GET

/api/lists/

List user's to do lists

POST

/api/lists/

Create a new list

POST

/api/lists/{id}/share/

Share list with another user

POST

/api/tasks/

Create a task in a list

Database Design

Recommended data structures and persistence model.

Entity 1

- id PK
- username
- email
- password_hash
- created_at

Entity 2

- id PK
- owner_id FK(User)
- title
- created_at

Entity 3

- id PK
- list_id FK(TodoList)
- content
- is_done
- created_at

Entity 4

- id PK
- list_id FK(TodoList)
- user_id FK(User)
- permission

Entity 5

- id PK
- user_id FK(User)
- token
- expires_at

Entity 6

- id PK
- action
- user_id FK(User)
- timestamp

Evaluation Criteria

Criteria that can be used to assess the completed project.

- Criterion: Correctness of core features | Description: All functional requirements work as specified | Weight: 30
- Criterion: Code quality & organization | Description: Clean, modular code with proper naming | Weight: 15
- Criterion: API design & documentation | Description: REST endpoints follow conventions and are documented | Weight: 10
- Criterion: Real time implementation | Description: WebSocket updates function reliably | Weight: 15

- Criterion: Testing coverage | Description: Unit, integration, and E2E tests present | Weight: 15

Additional Project Details

Testing Plan

- Unit tests for models (validation, relationships)
- API tests for authentication and CRUD endpoints
- WebSocket integration tests for real time updates
- React component snapshot tests
- Cypress end to end flow covering login, list sharing, and live updates
- Security tests for unauthorized access attempts

Implementation Note

This document is generated from the project record. AI-generated requirements should be reviewed by an administrator before implementation.